

Interview Prep: 50 String Problems (Complete Master Code)

Pattern 1 – Single Loop (Basic Iteration & ASCII)

java

```
public class StringSingleLoop {
    public static void main(String[] args) {
        String str = "Hello Java 123";
        char[] arr = str.toCharArray(); // String- Array

        // 1 & 7: Counting & Length without length()
        int vowels = 0, cons = 0, digits = 0, spaces = 0, length = 0;
        for (char ch : arr) {
            length++;
            if (ch == ' ') spaces++;
            else if (ch >= '0' && ch <= '9') digits++;
            else if (Character.toLowerCase(ch) == 'a' ||
Character.toLowerCase(ch) == 'e' ||
Character.toLowerCase(ch) == 'i' ||
Character.toLowerCase(ch) == 'o' ||
Character.toLowerCase(ch) == 'u') {
                vowels++;
            } else if (Character.isLetter(ch)) {
                cons++;
            }
        }

        // 2, 3, 4: Case Conversions (Difference is 32)
        StringBuilder upper = new StringBuilder();
        StringBuilder lower = new StringBuilder();
        StringBuilder toggle = new StringBuilder();

        for (char ch : arr) {
            if (ch >= 'a' && ch <= 'z') upper.append((char)(ch - 32));
            else upper.append(ch);

            if (ch >= 'A' && ch <= 'Z') lower.append((char)(ch + 32));
            else lower.append(ch);

            if (ch >= 'a' && ch <= 'z') toggle.append((char)(ch - 32));
            else if (ch >= 'A' && ch <= 'Z') toggle.append((char)(ch + 32));
        }
    }
}
```

```

        else toggle.append(ch);
    }

    // 5, 6, 8: Extract / Remove
    StringBuilder noSpace = new StringBuilder();
    StringBuilder noVowels = new StringBuilder();
    StringBuilder onlyDigits = new StringBuilder();

    for (char ch : arr) {
        if (ch != ' ') noSpace.append(ch);
        if (ch >= '0' && ch <= '9') onlyDigits.append(ch);

        char lowerCh = Character.toLowerCase(ch);
        if (lowerCh != 'a' && lowerCh != 'e' && lowerCh != 'i' &&
lowerCh != 'o' && lowerCh != 'u') {
            noVowels.append(ch);
        }
    }
}

## Pattern 2 – Frequency & Counting (Hashing Array)
java
public class StringFrequency {
    public static void main(String[] args) {
        String str = "programming";
        int[] freq = new int[256];
        for (int i = 0; i < str.length(); i++) freq[str.charAt(i)]++;

        int maxFreq = 0; char mostFreqChar = ' ';
        for (int i = 0; i < str.length(); i++) {
            char ch = str.charAt(i);

            // 9 & 10: Find Duplicates & Unique characters
            // if (freq[ch] > 1) { /* Print duplicate */ }
            // if (freq[ch] == 1) { /* Print unique */ }

            // 11. Most Frequent Character
            if (freq[ch] > maxFreq) { maxFreq = freq[ch]; mostFreqChar = ch; }
        }
    }
}

```

```

// 12. First Non-Repeating Character
for (int i = 0; i < str.length(); i++) {
    if (freq[str.charAt(i)] == 1) { /* Found first non-repeating */ break; }
}

// 15. Remove Duplicate Characters
StringBuilder removeDup = new StringBuilder();
boolean[] seen = new boolean[256];
for (int i = 0; i < str.length(); i++) {
    char ch = str.charAt(i);
    if (!seen[ch]) { removeDup.append(ch); seen[ch] = true; }
}

// 13. Check Anagram
String s1 = "listen", s2 = "silent";
int[] anagramFreq = new int[256];
for (char c : s1.toCharArray()) anagramFreq[c]++;
for (char c : s2.toCharArray()) anagramFreq[c]--;
boolean isAnagram = true;
for (int count : anagramFreq) { if (count != 0) isAnagram = false; }
}
}

## Pattern 3 – Two Pointers / Rotation
java
public class StringTwoPointers {
    public static void reverseCharArray(char[] arr, int start, int end) {
        while (start < end) {
            char temp = arr[start]; arr[start] = arr[end]; arr[end] = temp;
            start++; end--;
        }
    }
}

public static void main(String[] args) {
    // 16. Reverse a String
    String str = "hello";
    char[] arr = str.toCharArray();
    reverseCharArray(arr, 0, arr.length - 1);
    String reversed = new String(arr);

    // 17. Check Palindrome

```

```

String pal = "madam";
int left = 0, right = pal.length() - 1;
boolean isPal = true;
while (left < right) {
    if (pal.charAt(left) != pal.charAt(right)) { isPal = false; break; }
    left++; right--;
}

```

```

// 18. Reverse words in a sentence
String sentence = "I love java";
String[] words = sentence.split(" ");
StringBuilder revSentence = new StringBuilder();
for (int i = words.length - 1; i >= 0; i--)
revSentence.append(words[i]).append(" ");

```

```

// 19. Reverse each word
StringBuilder revEachWord = new StringBuilder();
for (String word : words) {
    char[] wordArr = word.toCharArray();
    reverseCharArray(wordArr, 0, wordArr.length - 1);
    revEachWord.append(new String(wordArr)).append(" ");
}

```

```

// 22. Check Rotation
String s1 = "ABCD", s2 = "CDAB";
boolean isRotation = (s1.length() == s2.length()) && (s1 +
s1).contains(s2);

```

```

// 20 & 21. Left & Right Rotate
int k = 2;
String leftRotated = s1.substring(k) + s1.substring(0, k);
String rightRotated = s1.substring(s1.length() - k) + s1.substring(0,
s1.length() - k);

```

```

// 23. Merge Two Strings Alternately
String m1 = "abc", m2 = "xyz";
StringBuilder merged = new StringBuilder();
int i = 0, j = 0;
while (i < m1.length() || j < m2.length()) {
    if (i < m1.length()) merged.append(m1.charAt(i++));

```

```

        if (j < m2.length()) merged.append(m2.charAt(j++));
    }
}
}

```

Pattern 4 – Sorting / Swap

```

``java
public class StringSortingSwap {
    public static void main(String[] args) {
        // 24. Sort a string alphabetically (Bubble Sort logic)
        String str = "dcba";
        char[] arr = str.toCharArray();
        for (int i = 0; i < arr.length - 1; i++) {
            for (int j = 0; j < arr.length - 1 - i; j++) {
                if (arr[j] > arr[j + 1]) {
                    char temp = arr[j]; arr[j] = arr[j + 1]; arr[j + 1] = temp;
                }
            }
        }

        // 25. Swap two strings without third variable
        String a = "Hello", b = "World";
        a = a + b;
        b = a.substring(0, a.length() - b.length());
        a = a.substring(b.length());
    }
}

```

Pattern 5 – Sliding Window

```

java
public class StringSlidingWindow {
    public static void main(String[] args) {
        // 26. Longest Substring Without Repeating Characters
        String s = "abcabcbb";
        int[] hash = new int[256];
        for (int i = 0; i < 256; i++) hash[i] = -1;

        int start = 0, maxLen = 0;
        for (int end = 0; end < s.length(); end++) {
            char ch = s.charAt(end);
            if (hash[ch] >= start) start = hash[ch] + 1; // Shrink window
        }
    }
}

```

```

        hash[ch] = end;
        maxlen = Math.max(maxlen, end - start + 1);
    }
}
}

```

Pattern 6 – Stack-Based (Brackets & Parsing)

```

``java
import java.util.Stack;
public class StringStackProblems {
    public static void main(String[] args) {
        // 30. Valid Parentheses
        String brackets = "{[()]}";
        Stack<Character> stack = new Stack<>();
        boolean isValid = true;
        for (char ch : brackets.toCharArray()) {
            if (ch == '(' || ch == '{' || ch == '[') stack.push(ch);
            else {
                if (stack.isEmpty()) { isValid = false; break; }
                char top = stack.pop();
                if ((ch == ')' && top != '(') || (ch == '}' && top != '{') || (ch == ']'
&& top != '[')) {
                    isValid = false; break;
                }
            }
        }
        if (!stack.isEmpty()) isValid = false;

        // 31. Remove outermost parentheses
        String s = "(()())()";
        StringBuilder res = new StringBuilder();
        int opened = 0;
        for (char c : s.toCharArray()) {
            if (c == '(' && opened++ > 0) res.append(c);
            if (c == ')' && opened-- > 1) res.append(c);
        }

        // 33. Reverse a string using Stack
        String str = "java";
        Stack<Character> revStack = new Stack<>();
    }
}

```

```

        for (char c : str.toCharArray()) revStack.push(c);
        StringBuilder reversed = new StringBuilder();
        while (!revStack.isEmpty()) reversed.append(revStack.pop());
    }
}

## Pattern 7 – Recursion (Backtracking)
java
public class StringRecursion {
    // 34. Print all permutations of a String
    public static void printPermutations(String str, String perm) {
        if (str.length() == 0) { System.out.println(perm); return; }
        for (int i = 0; i < str.length(); i++) {
            char currChar = str.charAt(i);
            String newStr = str.substring(0, i) + str.substring(i + 1);
            printPermutations(newStr, perm + currChar);
        }
    }

    // 35. Print all subsequences of a String
    public static void printSubsequences(String str, int idx, String
newString) {
        if (idx == str.length()) { System.out.println(newString); return; }
        printSubsequences(str, idx + 1, newString + str.charAt(idx)); //
Include
        printSubsequences(str, idx + 1, newString);                // Exclude
    }

    // 36. Check Palindrome using recursion
    public static boolean isPalRec(String s, int left, int right) {
        if (left >= right) return true;
        if (s.charAt(left) != s.charAt(right)) return false;
        return isPalRec(s, left + 1, right - 1);
    }

    // 37. Reverse string using recursion
    public static String reverseRec(String s) {
        if (s.isEmpty()) return s;
        return reverseRec(s.substring(1)) + s.charAt(0);
    }
}

```

Pattern 8 – Core Concepts (Comparison & Formatting)

java

```
public class StringCoreImplementations {
    public static void main(String[] args) {
        // 38. Compare two strings without equals()
        String s1 = "hello", s2 = "hello";
        boolean isEqual = s1.length() == s2.length();
        if (isEqual) {
            for (int i = 0; i < s1.length(); i++) {
                if (s1.charAt(i) != s2.charAt(i)) { isEqual = false; break; }
            }
        }

        // 39. Implement own substring(start, end)
        String mainStr = "programming";
        StringBuilder sub = new StringBuilder();
        for (int i = 3; i < 7; i++) sub.append(mainStr.charAt(i));

        // 40. Implement own indexOf()
        String text = "hello world", pattern = "world";
        int index = -1;
        for (int i = 0; i <= text.length() - pattern.length(); i++) {
            int j;
            for (j = 0; j < pattern.length(); j++) {
                if (text.charAt(i + j) != pattern.charAt(j)) break;
            }
            if (j == pattern.length()) { index = i; break; }
        }

        // 41. Convert String to Integer
        String numStr = "1234";
        int num = 0;
        for (int i = 0; i < numStr.length(); i++) num = num * 10 +
(numStr.charAt(i) - '0');

        // 42. Convert Integer to String
        int number = 4567;
        StringBuilder strNum = new StringBuilder();
        while (number > 0) { strNum.append((char)((number % 10) + '0'));
number /= 10; }
```

```

    strNum.reverse();

    // 43. Check if a string contains only digits
    String checkStr = "9876";
    boolean onlyDigits = true;
    for (char c : checkStr.toCharArray()) {
        if (c < '0' || c > '9') { onlyDigits = false; break; }
    }
}

## Pattern 9 – Special String Algorithms
java
public class StringSpecialAlgorithms {
    // 44. Longest Palindromic Substring
    public static int expandFromMiddle(String s, int left, int right) {
        while (left >= 0 && right < s.length() && s.charAt(left) ==
s.charAt(right)) {
            left--; right++;
        }
        return right - left - 1;
    }

    public static void main(String[] args) {
        // 45. Longest Common Prefix
        String[] strs = {"flower", "flow", "flight"};
        String prefix = strs[0];
        for (int i = 1; i < strs.length; i++) {
            while (strs[i].indexOf(prefix) != 0) prefix = prefix.substring(0,
prefix.length() - 1);
        }

        // 46 & 47. String Compression / Run Length Encoding
        String compStr = "aabcccccaaa";
        StringBuilder comp = new StringBuilder();
        int count = 1;
        for (int i = 0; i < compStr.length(); i++) {
            if (i + 1 < compStr.length() && compStr.charAt(i) ==
compStr.charAt(i + 1)) count++;
            else { comp.append(compStr.charAt(i)).append(count); count = 1; }
        }
    }
}

```

```
// 48. Find the largest word in a string
String sentence = "I am preparing for interview";
String[] words = sentence.split(" ");
String largestWord = "";
for (String w : words) if (w.length() > largestWord.length())
largestWord = w;
```

```
// 49. Capitalize the first letter of each word
StringBuilder capSentence = new StringBuilder();
boolean capNext = true;
for (char c : sentence.toCharArray()) {
    if (c == ' ') { capNext = true; capSentence.append(c); }
    else if (capNext)
{ capSentence.append(Character.toUpperCase(c)); capNext = false; }
    else { capSentence.append(c); }
}
```

```
// 50. Valid Shuffle of two strings
String s1 = "XY", s2 = "12", res = "1XY2";
int i = 0, j = 0, k = 0;
boolean isShuffle = (s1.length() + s2.length() == res.length());
if (isShuffle) {
    while (k < res.length()) {
        if (i < s1.length() && s1.charAt(i) == res.charAt(k)) i++;
        else if (j < s2.length() && s2.charAt(j) == res.charAt(k)) j++;
        else { isShuffle = false; break; }
        k++;
    }
}
}
```